



Department of Electronics & Communications

25SC2107E – Machine Learning

Practical Session

Gradient Descent from Scratch — CGPA Regression

Dataset: PlacementPredict | Course Outcome: CO2 | Environment: VS Code

Prepared by: **Dr. Omprakash Gottam**, Dept. of ECE, KLEF Hyderabad

1 Experiment 3: Gradient Descent from Scratch — CGPA Regression

Dataset: PlacementPredict

Course Outcome: CO2

1.1 Aim

To implement batch gradient descent from first principles in NumPy for the prediction of student CGPA from academic and co-curricular attributes; to verify the resulting parameter estimates against the closed-form solution produced by `LinearRegression`; and to extend the implementation to mini-batch gradient descent, comparing the convergence behaviour and predictive error of the two variants.

1.2 Learning Objectives

After completing this experiment, students should be able to:

- Write the gradient descent update rule in NumPy without using scikit-learn.
- Plot and interpret an MSE loss curve, and name the fault behind each failure shape.
- Verify a hand-written optimiser against a trusted reference implementation.
- Explain the difference between batch and mini-batch gradient descent.
- Use `assert` to state a numerical tolerance a result must satisfy.

1.3 Environment and Libraries

VS Code with the Python + Jupyter extensions, `.venv` activated.

```
pip install numpy pandas matplotlib scikit-learn
```

1.4 Dataset and Link

`PlacementPredict` is the course dataset, supplied as `placement_predict_50k.csv` in this folder. Keep the file beside your notebook.

- **50,000 rows**, one per student.
- **Numeric columns:** CGPA, Internships, Projects, AptitudeScore, SoftSkillsRating, Backlogs.
- **Text columns:** Stream, Gender, PlacementTraining, CGPA_Tier, PlacementStatus.
- **Nothing is missing.** There is no cleaning to do, which is deliberate. The new idea in this experiment is the *algorithm*, not the data.

In this experiment the answer column is CGPA, a number, so this is a **regression** problem. The four inputs are Internships, Projects, AptitudeScore, SoftSkillsRating and Backlogs.

Note

Why write gradient descent by hand at all?

- Gradient descent is the method that trains almost every model you will meet, including neural networks.
- Writing it once, in about ten lines, removes the mystery permanently.
- You then check your version against scikit-learn. If the two agree, you have proved to yourself that the library is doing exactly what you just wrote, only faster.

1.5 Procedure

1. Create a notebook `tut_lab_LR_GD.ipynb` in this folder.
2. Load `placement_predict_50k.csv` and inspect its structure.

3. Choose the input columns and the answer column, then split into training and validation sets.
4. Scale the inputs. (Gradient descent will not work without this step).
5. Fit `LinearRegression` to obtain the reference answer.
6. Write batch gradient descent in NumPy and plot its loss curve.
7. Compare the hand-written with scikit-learn's.
8. Write mini-batch gradient descent, plot both curves together, and observe the tolerance.

1.6 Notebook Implementation

Cell 1 — Import the libraries

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from sklearn.model_selection import train_test_split
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.linear_model import LinearRegression
7 from sklearn.metrics import mean_squared_error, r2_score
```

What is happening:

- `numpy` does the maths. In this experiment it does the *training* as well, which is new.
- `pandas` holds the table and `matplotlib` draws the loss curves.
- `LinearRegression` is imported only as a **reference answer** to check your own work against, not as the thing being taught.

Cell 2 — Load the data

```
1 placement = pd.read_csv("placement_predict_50k.csv")
2 print(placement.shape)
3 placement.head()
```

What is happening:

- The file sits in the same folder as the notebook, so a plain filename is enough. No web address is needed.
- `shape` prints (50000, 12): fifty thousand students, twelve columns.
- Look at `head()` and find the `CGPA` column. That is what you are going to predict.

Cell 3 — Check the structure

```
1 placement.info()
2 print(placement.describe())
```

What is happening:

- `info()` shows the type of every column and how many values are filled. Every count reads 50,000, so nothing is missing.
- `describe()` gives the smallest, largest and average of each number column.
- Confirm that `CGPA` runs from about 5.0 to 10.0. Knowing that range matters: an error of 0.5 in a `CGPA` prediction is large, whereas an error of 0.5 in a house price measured in lakhs would be small. Error is only meaningful against the scale of the answer.

Cell 4 — Choose the columns and split

```
1 features = ["Internships", "Projects", "AptitudeScore",
2            "SoftSkillsRating", "Backlogs"]
3
4 X = placement[features]
5 y = placement["CGPA"]
```

```

6
7 X_train, X_val, y_train, y_val = train_test_split(
8     X, y, test_size=0.2, random_state=42)
9 print(X_train.shape, X_val.shape)

```

What is happening:

- Five input columns are used. CGPA itself is the answer, so it must never appear in X.
- `test_size=0.2` gives **40,000** rows to learn from and **10,000** to be tested on.
- `random_state=42` keeps the shuffle identical between runs.

Cell 5 — Scale the inputs

```

1 scaler = StandardScaler()
2 X_train_s = scaler.fit_transform(X_train)
3 X_val_s = scaler.transform(X_val)
4 print(X_train_s.mean().round(3), X_train_s.std().round(3))

```

What is happening:

- Same rule as always: **fit on train, transform on both.**
- The printout confirms it: mean 0, spread 1.

Here scaling is not a nicety. Without it the next cell simply fails.

- AptitudeScore runs to 100 while SoftSkillsRating runs to 5.
- Gradient descent uses **one step size for every input at once.**
- A step small enough to be safe for AptitudeScore barely moves SoftSkillsRating, so the loop never arrives.
- A step big enough to move SoftSkillsRating sends AptitudeScore past the target and further away each time, until the numbers become NaN.
- Scaling puts all five inputs on the same footing, so a single step size suits them all.

Cell 6 — The reference answer

```

1 lr = LinearRegression()
2 lr.fit(X_train_s, y_train)
3
4 sk_pred = lr.predict(X_val_s)
5 sk_rmse = np.sqrt(mean_squared_error(y_val, sk_pred))
6 print("sklearn RMSE:", round(sk_rmse, 4))
7 print("sklearn R2  :", round(r2_score(y_val, sk_pred), 4))

```

What is happening:

- This is the target you must hit, not the lesson itself.
- **RMSE** is about 0.51. Since CGPA runs from 5 to 10, the model is typically wrong by about half a grade point.
- R^2 is about 0.74, so roughly three quarters of the variation in CGPA is explained by the five inputs.
- Scikit-learn does not use gradient descent here. It solves the equations directly in one step. Remember that: the two methods reach the same place by different roads.

Note**The two scores, explained properly**

RMSE — root mean squared error. It answers: *by how much is the model typically wrong?*

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

- Read it from the inside out. Take each mistake $y_i - \hat{y}_i$, square it so that negatives cannot

cancel positives, average the squares, then take the square root to return to the original units.

- That last square root is why we bother: it puts the answer back into **CGPA points**. An RMSE of 0.51 means a typical prediction is about half a grade point out.
- Squaring before averaging means one large mistake counts far more than several small ones.
- **Smaller is better**, and 0 is perfect. There is no upper limit.

R^2 — **the coefficient of determination**. It answers: *how much better is this model than simply guessing the average?*

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} = 1 - \frac{\text{error our model makes}}{\text{error the average-guesser makes}}$$

- The bottom line is the error you would make by ignoring every input and always predicting the mean CGPA. That is the **baseline**, exactly like the 62% baseline in Experiment 1.
- The top line is the error your model actually makes.
- So the fraction is “how much of the baseline error is left over”, and R^2 is what you removed.

R^2	Meaning
1.0	perfect: every prediction exactly right
0.74	our model: about three quarters of the variation explained
0.0	no better than always guessing the average
negative	worse than guessing the average

Why report both? R^2 has no units, so it travels between problems: 0.74 means the same thing for CGPA as for house prices. RMSE has units, so it is the one to quote to a person: “typically wrong by half a grade point” is understandable in a way that “ $R^2 = 0.74$ ” is not.

Cell 7 — Batch gradient descent, written by hand

```

1 def batch_gd(X, y, epochs=1000, rate=0.1):
2     w = np.zeros(X.shape[1])
3     b = 0.0
4     y = np.asarray(y, dtype=float)
5     losses = []
6
7     for step in range(epochs):
8         error = X @ w + b - y
9         losses.append((error ** 2).mean())
10        w = w - rate * (X.T @ error) / len(y)
11        b = b - rate * error.mean()
12
13    return w, b, losses
14
15
16 w_batch, b_batch, loss_batch = batch_gd(X_train_s, y_train)
17 print("Final MSE:", round(loss_batch[-1], 4))

```

What is happening: read it one line at a time. This is the entire training algorithm.

- w holds one weight per input column; b is the starting height. Both begin at zero, meaning “predict nothing”.
- $X @ w + b$ predicts all 40,000 students at once. The symbol $@$ means matrix multiplication.
- $error$ is how wrong each prediction is. A positive value means the prediction was too high.
- The mean squared error is stored so it can be plotted afterwards.

- The two update lines move w and b a small distance downhill, in the direction that makes the error smaller. One execution of those two lines is called **one update**.
- The loop repeats 1000 times. Each full pass over the data is called an **epoch**.

The word “batch” means the whole training set is used in every single step. All 40,000 rows contribute to each update. That is what makes each update accurate but slow to compute.

Note

Three words that get used loosely. Fix them now.

Word	What it means, exactly
one update	One execution of the two lines that change w and b . The parameters move once. Also called one <i>step</i> or one <i>iteration</i> .
one epoch	One complete pass over the training data, so that every row has been used once.
batch size	How many rows are used to compute a single update.

The relationship between them:

$$\text{updates in one epoch} = \frac{\text{number of rows}}{\text{batch size}}$$

- In this cell the batch size *is* the whole training set, so one epoch contains exactly **one** update. After 1000 epochs the parameters have moved 1000 times.
- In the deliverable the batch size is 32, so one epoch contains $40,000/32 = 1,250$ updates, and 1000 epochs give 1.25 million.
- **An epoch is not an update.** That is the confusion this box exists to prevent.

Cell 8 — The loss curve

```
1 plt.plot(loss_batch)
2 plt.xlabel("epoch")
3 plt.ylabel("mean squared error")
4 plt.title("Batch Gradient Descent")
5 plt.show()
```

What is happening: this is the picture that tells you whether training worked.

- It should fall steeply at first, then flatten.
- Flattening means the algorithm has **converged**: further epochs would change almost nothing.

The shape names the fault. Learn all four.

- **Falls, then flattens** → correct. This is what you want.
- **Still falling steeply at epoch 1000** → stopped too early, or **rate** is too small.
- **Rises, wobbles, or becomes NaN** → **rate** is too large.
- **Flat from the very first epoch** → the inputs were not scaled.

Cell 9 — Did the hand-written version work?

```
1 gd_pred = X_val_s @ w_batch + b_batch
2 gd_rmse = np.sqrt(mean_squared_error(y_val, gd_pred))
3
4 print("sklearn RMSE      :", round(sk_rmse, 4))
5 print("batch GD RMSE     :", round(gd_rmse, 4))
6 print("largest coef gap :", round(np.abs(w_batch - lr.coef_).max(), 6))
```

What is happening:

- The two RMSE values are identical to four decimal places, and the largest difference between any pair of coefficients is essentially zero.
- **This is the result the experiment exists to produce.** Ten lines of NumPy reproduce a library function exactly.
- Note carefully what has been proved and what has not. You have not shown that your code is *fast*; scikit-learn is far faster. You have shown that it is *correct*.
- Read the weights themselves as well: Backlogs carries a negative weight, and the other four are positive. That matches common sense, which is a further check that nothing has gone wrong.

Deliverable Function

```

1 def minibatch_gd(X, y, epochs=1000, batch_size=32, rate=0.01):
2     rng = np.random.default_rng(42)
3     w = np.zeros(X.shape[1])
4     b = 0.0
5     y = np.asarray(y, dtype=float)
6     n = len(y)
7     losses = []
8
9     for step in range(epochs):
10        order = rng.permutation(n)           # reshuffle each epoch
11        for start in range(0, n, batch_size):
12            idx = order[start:start + batch_size]
13            error = X[idx] @ w + b - y[idx]
14            w = w - rate * (X[idx].T @ error) / len(idx)
15            b = b - rate * error.mean()
16            losses.append(((X @ w + b - y) ** 2).mean())
17
18        return w, b, losses
19
20
21 w_mini, b_mini, loss_mini = minibatch_gd(X_train_s, y_train)
22
23 mini_rmse = np.sqrt(mean_squared_error(y_val, X_val_s @ w_mini +
24                                     b_mini))
25 print("sklearn RMSE      :", round(sk_rmse, 4))
26 print("mini-batch RMSE :", round(mini_rmse, 4))
27
28 assert abs(mini_rmse - sk_rmse) < 0.3, "mini-batch RMSE is not within
29     0.3 of sklearn"
30 print("Tolerance check PASSED")
31
32 plt.plot(loss_batch, label="batch")
33 plt.plot(loss_mini, label="mini-batch")
34 plt.xlabel("epoch")
35 plt.ylabel("mean squared error")
36 plt.legend()
37 plt.title("Batch vs Mini-batch")
38 plt.show()

```

What is happening: the only change is *how much data is used per update*.

- **Batch** used all 40,000 rows for each update, giving 1000 updates in total.
- **Mini-batch** uses only 32 rows per update. With 40,000 rows that is 1,250 updates *per epoch*, and 1.25 million updates overall.
- `rng.permutation(n)` reshuffles the students at the start of every epoch, so the batches

differ each time. Without reshuffling the model would see the same 32 students together on every pass and could learn their peculiarities.

- The inner loop walks through the shuffled order in steps of 32.
- The loss is recorded once per *epoch*, not once per batch, so the two curves can be plotted on the same axis.

Why bother, when batch already gave the exact answer?

- Each mini-batch update is computed from 32 rows instead of 40,000, so it is over a thousand times cheaper.
- That matters enormously when a dataset is too large to fit in memory at once. Mini-batch is how every large model is really trained.
- The price is accuracy per step: 32 students are a noisy sample of 40,000, so the path downhill wobbles rather than running straight.
- Look at the two curves together. The mini-batch curve is slightly rougher, but it drops faster in the earliest epochs, because it makes 1,250 updates in the time batch makes one.

The assert line states what “close enough” means.

- Mini-batch is random, so it will never match scikit-learn to the last decimal, and demanding that it should would be wrong.
- Instead you declare a tolerance: the RMSE must land within 0.3 of the reference.
- If it does, nothing is printed and the cell moves on. **Silence means success.**
- If it does not, the cell stops with your message. That is a signal to check the learning rate or the number of epochs.

1.7 Expected Output

Console:

```
(50000, 12)
(40000, 5) (10000, 5)
sklearn RMSE: 0.5106
sklearn R2   : 0.7448
Final MSE: 0.2618
sklearn RMSE      : 0.5106
batch GD RMSE     : 0.5106
largest coef gap  : 0.0
sklearn RMSE      : 0.5106
mini-batch RMSE   : 0.5107
Tolerance check PASSED
```

The mini-batch cell takes roughly ten to fifteen seconds to run. That is expected: it performs 1.25 million parameter updates.

1.8 Result

Batch gradient descent was implemented in NumPy and applied to the prediction of student CGPA from five academic and co-curricular attributes. Over 1000 epochs at a learning rate of 0.1 the procedure converged to a root-mean-square error of 0.5106 on the held-out validation partition of 10,000 records, and the resulting coefficient vector agreed with the closed-form solution obtained from `LinearRegression` to within 10^{-6} , confirming the correctness of the implementation. The loss curve exhibited the expected monotonic decrease followed by convergence. The implementation was subsequently extended to mini-batch gradient descent with a batch size of 32 over 1000 epochs, comprising approximately 1.25 million parameter updates, which attained a validation root-mean-square error of 0.5107 and satisfied the prescribed tolerance of 0.3 relative to the reference implementation. Comparison of the two loss trajectories demonstrated the characteristic trade-off between the two variants: batch descent produces a smooth descent path at high cost per step, whereas mini-batch descent produces a noisier trajectory at substantially lower cost per update.

Note**Batch and mini-batch, side by side**

	Batch	Mini-batch
Rows used per update	all 40,000	32
Updates per epoch	1	1,250
Cost of one update	high	very low
Path downhill	smooth	wobbly
Fits in memory?	only if data is small	always

There is a third variant you will meet later. Using *one* row per update is called **stochastic** gradient descent. Mini-batch sits between the two extremes, which is why it is the one used in practice.

Self-Exploration & Viva Questions

1. What exactly does one *epoch* mean, and how many parameter updates does it contain for each of the two methods?
2. Why does `rng.permutation` appear inside the epoch loop rather than before it? What could go wrong if it were moved outside?
3. *Explore:* set `rate = 0.5` and then `rate = 0.0001` in Cell 7. Plot both loss curves and label which fault each one shows.
4. *Explore:* run `minibatch_gd` with `batch_size=1` and then `batch_size=4096`. Which resembles the batch curve, and why?